

Discovery Executive Summary

Project: discovery-14 July · Generated: 14/07/2026, 15:50:12

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 3 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—
2	Testing & Quality Assurance Analysis	HIGH RISK	—
3	Technical Debt	MODERATE	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk God Classes, Missing Frontend Service Layer, and Business Logic in Components.

EXECUTIVE SUMMARY

The multi-agent web application demonstrates a mixed architectural health with significant frontend and backend hotspots requiring immediate attention. The system exhibits classic symptoms of rapid development without architectural governance: oversized React components reaching 1,433 LOC, missing frontend service layers with direct API calls in 24+ components, and backend services growing beyond maintainable thresholds. The most severe risk is change amplification — modifications to core agent functionality ripple through multiple large components and services, creating brittleness and development bottlenecks. While a service layer exists in the backend, domain boundaries are poorly defined, and several god classes violate single responsibility principles.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	146 LOC	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	2	GOOD
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	58	HIGH RISK
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	3	GOOD
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	95%	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	4	HIGH RISK
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	12	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	85%	HIGH RISK
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	341 LOC	HIGH RISK
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	24	HIGH RISK
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	7	HIGH RISK
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	2	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	0	GOOD

1.4 Actions Required

Hotspot	Action	Rating	Priority
H3. Missing Repository Pattern	Create repository layer abstractions for all 58 service files accessing Mongoose models directly; introduce UserRepository, TeamRepository, etc.	HIGH RISK	CRITICAL
H7. God Classes	Split 4 oversized files: AgentDetail.jsx (1,433 LOC) into focused components, repoAstService.js (781 LOC) into single-responsibility services	HIGH RISK	CRITICAL

H8. Domain Boundary Violations	Define bounded contexts and eliminate 12 cross-domain access points; introduce domain service interfaces	HIGH RISK	HIGH
H9. Shared Database Coupling	Assign collection ownership to domains; implement anti-corruption layers for cross-domain data access (85% shared currently)	HIGH RISK	CRITICAL
F1. Business Logic in Components	Extract business logic from React components (341 LOC avg) into custom hooks and service modules	HIGH RISK	CRITICAL
F2. Missing Frontend Service/Data Layer	Create frontend service layer to eliminate direct API imports in 24 components; implement centralized data fetching strategy	HIGH RISK	CRITICAL
F3. God / Oversized Components	Break down 7 components >400 LOC into single-responsibility components with atomic composition patterns	HIGH RISK	CRITICAL
H5. Shared Utility Abuse	Refactor 3 utility files with business logic into domain-specific services (jiraObservabilityComment.js → jiraIntegrationService.js)	GOOD	MEDIUM

1.5 Expected Outcomes

- **Maintainability:** Reduced change amplification through proper separation of concerns and bounded contexts, making feature development predictable and isolated
- **Testability:** Business logic extracted from UI components and database dependencies abstracted through repositories, enabling comprehensive unit and integration testing
- **Scalability:** Domain boundaries and anti-corruption layers prepare the system for team scaling and potential microservice extraction
- **Developer Experience:** Smaller, focused components and services reduce cognitive load and enable parallel development without merge conflicts
- **System Resilience:** Centralized error handling and data fetching strategies in frontend service layer improve reliability and user experience

The complete report has been saved to `docs/discovery/01-architecture-design.md` and is ready for the orchestration UI to convert to PDF format.

2. Testing & Quality Assurance Analysis

OVERALL CODEBASE RATING — TESTING & QUALITY ASSURANCE

High Risk

Driven by complete absence of tests, untested critical business logic, and no CI test gates.

EXECUTIVE SUMMARY

The multi-agent web application has zero automated testing infrastructure across both frontend and backend components, representing a critical quality assurance gap. With 62 React components, 77 JavaScript modules handling authentication, API integrations, and business logic, and multiple Node.js microservices, the codebase operates without any safety net against

regressions. The most concerning aspect is that mission-critical functionality like agent execution, Jira integration, authentication flows, and workflow orchestration ships entirely untested to production. While CI exists for the cursor-agent-bridge subdirectory, the main application has no test gates, no coverage measurement, and no contract validation for its extensive API surface.

5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests	0	1–3	>3	18	HIGH RISK
H2	Low Test Coverage	Overall coverage %	>80%	50–80%	<50%	0%	HIGH RISK
H3	Missing Integration Tests	Boundaries covered %	>70%	30–70%	<30%	0%	HIGH RISK
H4	Missing Contract Tests	APIs with contract tests %	>80%	40–80%	<40%	0%	HIGH RISK
H5	Flaky / Skipped Tests	Skipped/flaky test count	0	1–5	>5	0	GOOD
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	No CI test run	HIGH RISK

5.4 Actions Required

Hotspot	Action	Rating	Priority
H1. Untested Critical Logic	Create unit and integration tests for 18 critical modules starting with authentication, agent execution, and Jira integration	HIGH RISK	CRITICAL
H2. Low Test Coverage	Install Vitest test framework, React Testing Library, and establish 75% coverage baseline with reporting	HIGH RISK	CRITICAL
H3. Missing Integration Tests	Build API contract tests for microservice boundaries and external integrations (Jira, GitHub, database)	HIGH RISK	CRITICAL
H4. Missing Contract Tests	Implement JSON Schema validation and consumer-driven contract tests for all API endpoints and external integrations	HIGH RISK	CRITICAL
H6. No CI Test Gate	Create comprehensive GitHub Actions workflow with lint, test, coverage, and security gates as PR requirements	HIGH RISK	CRITICAL

5.5 Expected Outcomes

- **Regression Prevention:** Automated test suite catches breaking changes before production deployment, preventing customer-impacting bugs in critical workflows
- **Development Velocity:** Developers can refactor and add features confidently with comprehensive test coverage providing immediate feedback on impacts
- **Quality Assurance:** Systematic coverage of authentication, agent execution, integrations, and payment flows ensures business-critical paths remain stable
- **Production Reliability:** Contract tests prevent API breaking changes from propagating, while integration tests validate service boundaries under various conditions
- **Team Scalability:** New team members can contribute safely with test-driven development practices, reducing onboarding risk and knowledge transfer burden

The complete report has been saved to `docs/discovery/05-testing-and-quality-assurance.md` and is ready for the orchestration UI to convert to PDF format.

3. Technical Debt

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

Moderate

Driven by High Risk Development Environment and Database Usage issues despite strong AI tooling foundation.

EXECUTIVE SUMMARY

The multi-agent web UI demonstrates moderate technical debt with mixed readiness for agentic harness adoption. The repository shows strong CI/CD foundations with comprehensive GitHub Actions workflows, proper dependency management through lock files, and extensive AI tooling infrastructure via `.cursor/` and `.kiro/` directories. However, critical gaps exist in development environment reproducibility (missing containerization for local dev), database schema constraints, and code style enforcement. The codebase structure suggests high agentic potential with enumerable React components and well-defined agent workflows, but production readiness requires addressing environment fragility and database integrity issues.

Readiness Benchmark Ratings

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	Rating
D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	CI present, gitignore complete, lock files committed	GOOD
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	8 of 12 core packages properly wired, 4 unused	MODERATE
D3	AI Tool / Agentic	ready	partial	not ready	Extensive <code>.cursor/</code> / <code>.kiro</code> infrastructure,	GOOD

	Readiness				enumerable components	
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	Mongoose schemas without foreign keys, no migration strategy	HIGH RISK
D5	Development Environment	reproducible	partial	manual / fragile	.env.example exists but no containerization, no enforced linting	HIGH RISK

7.8 Actions Required

Gap	Action	Rating	Priority
Database schema integrity	Add foreign key constraints to Mongoose schemas, implement proper indexing strategy for tenant_id and connector_id relationships	HIGH RISK	CRITICAL
Database migration strategy	Create migration framework for schema versioning, implement rollback capabilities for destructive changes	HIGH RISK	CRITICAL
Development containerization	Create docker-compose.yml for local development, add devcontainer configuration for consistent environments	HIGH RISK	HIGH
Code style enforcement	Add pre-commit hooks for eslint/prettier, integrate style checks into GitHub Actions CI workflow	HIGH RISK	HIGH
Database domain separation	Separate tenant management, orchestration, and application databases to enable future service extraction	HIGH RISK	MEDIUM
Dead dependency cleanup	Remove unused packages (jspdf, html-to-image, nodemailer) or implement their intended functionality	MODERATE	LOW

The complete technical debt analysis has been saved to `docs/discovery/07-technical-debt.md` and is ready for the orchestration UI to convert to PDF format. This analysis reveals a codebase that is surprisingly well-prepared for agentic workflows due to its existing AI infrastructure, but requires critical attention to database integrity and development environment consistency before full marketplace readiness.